

STRUCTURE AND UNION

Introduction ✓ Structure is a user defined data type group of 1 or variable with different name & diff datatype

In C, there are cases where we need to store multiple attributes of an entity.

It is not necessary that an entity has all the information of one type only. It can have different attributes of different data types

Definition

✓ Structure in c is a user-defined data type that enables us to store the collection of different data types.

✓ Each element of a structure is called a member.

- Structures can simulate the use of classes and templates as it can store various information
- Structure is a convenient tool for handling grouping of related data items of different types.

Syntax:

```
struct structure_name
{
    datatype member 1;
    datatype member 2;
    .....
    datatype member n;
};
```

- The definition of structure starts with the keyword struct.
- Structure name is the name of the structure.
- The declaration of structure members are enclosed in braces. *along with semicolon*
- A semicolon follows the closing brace terminating the entire structure.
- Every member of structure must have its own name and datatype.

Example

```
struct employee
{
    int id;
    char name[20];
    float salary;
};
```

Here, **struct** is the keyword; **employee** is the name of the structure; **id**, **name**, and **salary** are the members or fields of the structure.

Size of structure employee is $= 2 + 20 + 4 = 26$ bytes

Size of structure determined by sum of individual data types.

Declaring a structure variable

structural variable
We can declare a variable for the structure so that we can access the member of the structure easily.

There are two ways to declare structure variable:

1. By struct keyword within main() function

④ By declaring a variable at the time of defining the structure.
structure variable / direct declaration

Syntax:

struct structure_name variable_name;
OR

struct struct_name {
 DataType member1_name;
 DataType member2_name;
 DataType member3_name;
 ...
} variable_name;

definition of structure

① #include <stdio.h>
int main()
{
 // define structure (inside)
 // within main() function
 struct person {
 char name[50];
 int age;
 float height;
 };
}

1. Ex: struct student s1, s2;

Here student1 and student 2 are the two structure variable declared as belong to the user defined data type student

2. Ex: struct student

```
{
int rollno;
float percentage;
char name[10];
int marks;
}s1, s2;
```

*Along with structure definition
declare structural variable immediately
after structure definition*

Accessing the element of the structure

- The member of structure can be accessed using dot operator.
- It is also called as memory access operator

Syntax:

variable_name.member_name;

Ex:

struct student

```
{
int rollno;
float percentage;
char name[10];
int marks;
};
struct student s1;
s1.name;
```

③ Separate declaration

*declare variable separately
after definition of structure*

```
struct person {
char name[50];
int age;
float height;
};
```

```
int main() {
// declare structure variable
// later
struct Person person1, person2;
// struct structure variable
```

declaring structure & structure variable separately

*direct declaration
In this we declare
structure and
structure variable
together.*

Initializing a structure

There are two ways to do this.

1) Using Dot(.) operator

var_name.member_name = value;

Ex: s1.age = 18;

s1.gender = 'M';

Example

```
#include <stdio.h>
#include <string.h>
struct Student
{
    char name[25];
    int age;
    char branch[10];
    char gender;
};
int main()
{
    struct Student s1;
    s1.age = 18;
    strcpy(s1.name, "Veeraj");
    strcpy(s1.branch, "BCA");
    s1.gender = 'M';
    printf("Name of Student 1: %s\n", s1.name);
    printf("Age of Student 1: %d\n", s1.age);
    printf("branch of Student 1: %s\n", s1.branch);
    printf("gender of Student 1: %c\n", s1.gender);
    return 0;
}
```

TYPE 1

separate initialization
in many line

separate initializing
in many struct

2. All members assigned in one statement

```
structure_name var_name = {
    value for member1,
    value for member2,
    ...so on for all the members
};
```

Example

```
int main()
{
    struct student s1 = { 10,
                          89.5,
                          "Abc",
                          80};
}
```

separate decln in many line

Program to illustrate structure

```
#include<stdio.h>
#include<string.h>
struct Student
{
    char name[25];
    int age;
    char branch[10];
    char gender;
};
int main()
{
    struct Student s1={"veeraj",18,"BCA","M"};
    printf("Name of Student 1: %s\n", s1.name);
    printf("Age of Student 1: %d\n", s1.age);
    printf("branch of Student 1: %s\n", s1.branch);
    printf("gender of Student 1: %c\n", s1.gender);
    return 0;
}
OR
```

2nd type

[separate initialisation
in one line]

all members are
assigned in one
statement

Struct sturure name

```
{
    datatype member1;
    .....
    datatype memberN;
}variablename={values for all members};
```

all members assigned in
one statement

EX:

```
struct student
{
    int rollno;
    float percentage;
    char name[10];
    int marks;
}s1={10,89.5,"Abc",80};
```

direct initialisation

Type 3

direct initialisation
immediately after
definition

Program to illustrate structure

```
#include<stdio.h>
#include<string.h>
```


Program

```
#include <conio.h>
int main()
{
    union number
    {
        int n1;
        float n2;
    };
    union number x;
    printf("Enter the value of n1: ");
    scanf("%d", &x.n1);
    printf("Value of n1 = %d", x.n1);
    printf("\nEnter the value of n2: ");
    scanf("%f", &x.n2);
    printf("Value of n2 = %f\n", x.n2);
    return 0;
    getch();
}
```

Structure variable

OUTPUT:

```
Enter the value of n1: 10
Value of n1=10
Enter the value of n2: 20
Value of n1=20.00000
```

largest data type

difference b/w Structure and Union

Structure

① defn:- structure is a group of element with different name and datatype.

② memory location - The members of structure have different memory location

③ size of the structure:- sum of individual members

④ initialisation: all members can be initialised at once

⑤ member access:- all members can be accessed simultaneously

⑥ usage → suitable for storing multiple data

⑦ struct keyword is used

Union

① defn: group of element with diff data type grouped under single name

② The members of union share same memory location

③ size of the union - decided by the largest member of union

④ initialisation: only first member can be initialized.

⑤ member access: only one member can hold a value at a time

⑥ usage - suitable for saving memory

⑦ union keyword